STUDENT RECORD MANAGEMENT SYSTEM

Project Report

Student Name	[Enter your name]
Roll Number	[Enter your roll number]
Course	Python Essentials
Project Type	Python application with Streamlit interface
Storage	Local JSON file

Submitted as part of the Python Essentials project evaluation.

1. Introduction

The Student Record Management System is a beginner-level application developed to practise core Python Essentials concepts in a practical context. It organizes basic student details and subject-wise marks. A Streamlit interface provides forms and navigation, while a command-line interface is also included. Student data is stored locally in a JSON file.

2. Problem Statement

Manual record keeping can make it difficult to consistently maintain student details, retrieve records, and calculate academic results. The project demonstrates a small software solution for registering students, recording marks, viewing records, and calculating a simple result summary.

3. Objectives

• Create and store student records with a unique student ID.

• Display student records in a readable table.

• Allow subject marks to be added or updated.

• Calculate total marks, average subject percentage, and a grade.

• Practise Python functions, loops, conditions, dictionaries, lists, modules, file handling, and validation.

4. Functional Requirements

ID	Requirement	Input	Output
FR-1	Register a student	ID, name, course, semester	Record saved in JSON
FR-2	View student records	User selects View Students	Student table or empty-state message
FR-3	Delete a student	Selected ID + confirmation	Record removed and data saved
FR-4	Record subject marks	Student, subject, score 0–100	Marks saved and displayed
FR-5	Calculate result	Saved subject scores	Total, average percentage, grade
FR-6	Persist records	Student/marks changes	JSON file updated

5. Non-Functional Requirements

• Usability: clear page names, labels, and form prompts.

• Reliability: changes are persisted to a local JSON file.

• Maintainability: functionality is separated across modules.

• Input validation: empty required fields, duplicate IDs, and score ranges are checked where implemented.

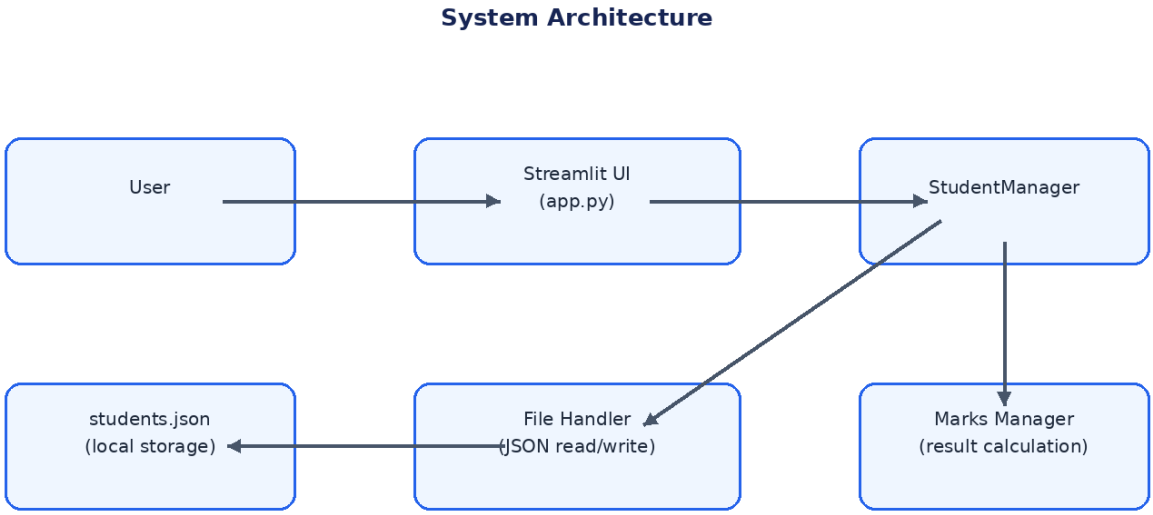
• Resource efficiency: local JSON storage and standard Python data structures support a small academic dataset.

• Portability: the project can run on a system with a compatible Python installation and installed requirements.

6. System Architecture

The application uses a simple layered structure. The user interacts through Streamlit or the command-line menu. The StudentManager coordinates record operations, marks_manager calculates results, and file_handler reads and writes data/students.json. student.py defines the record structure; validators.py contains reusable command-line input checks.

Figure 1. System architecture



7. Design Diagrams

Figure 2. Application workflow

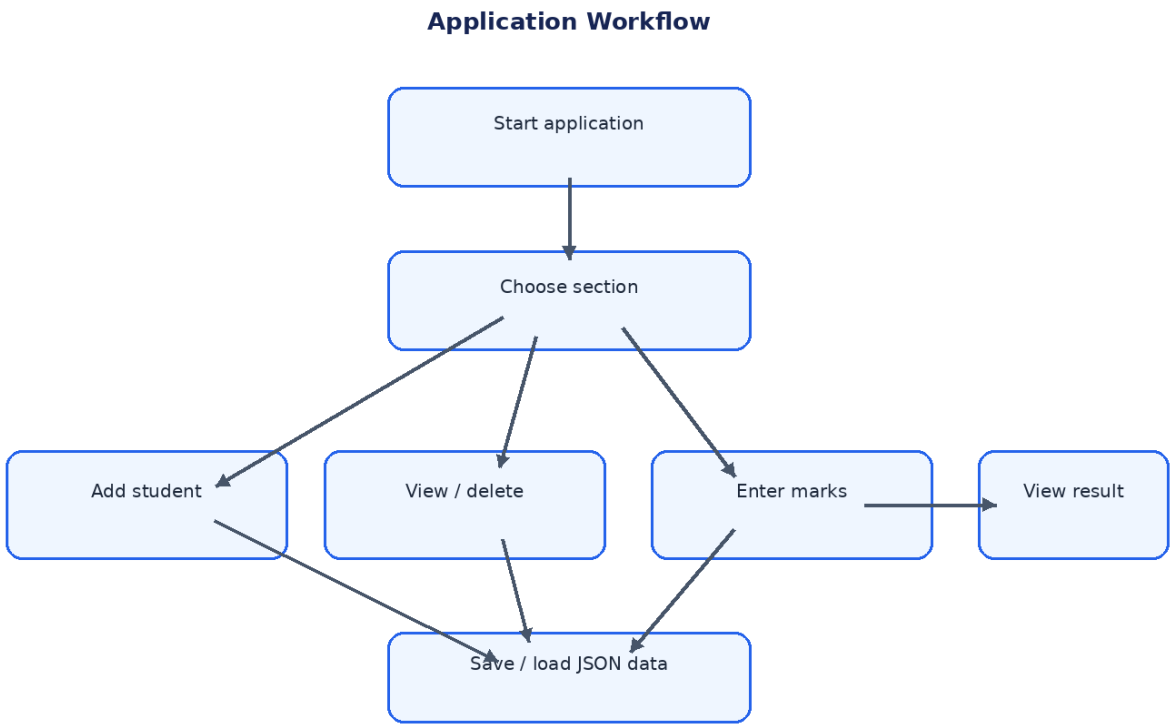


Figure 3. Use case diagram

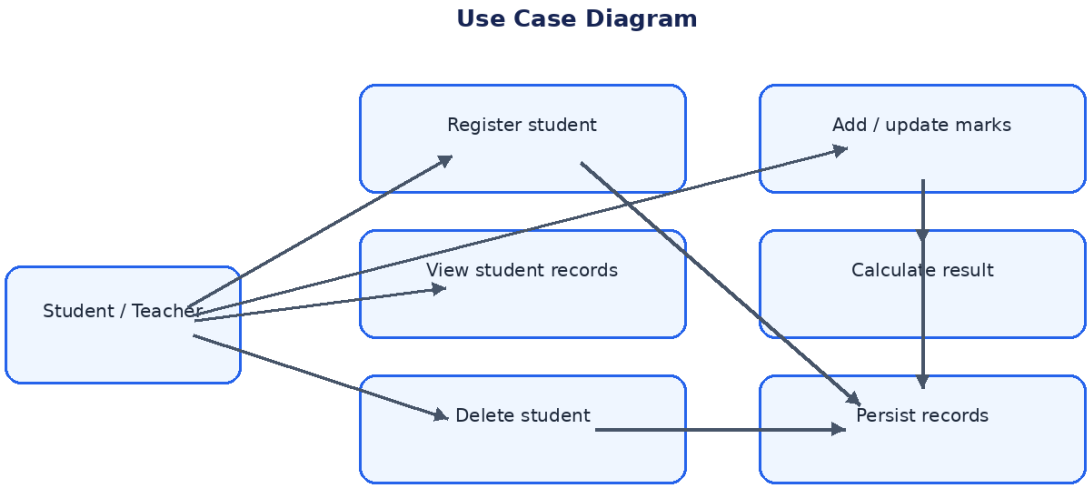


Figure 4. Component diagram

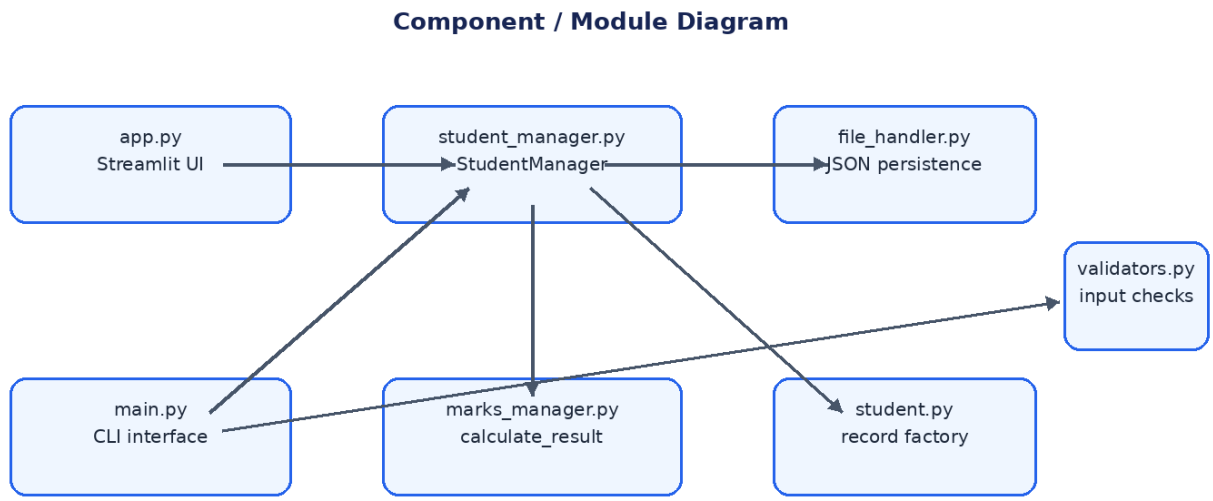


Figure 5. Sequence diagram: saving marks

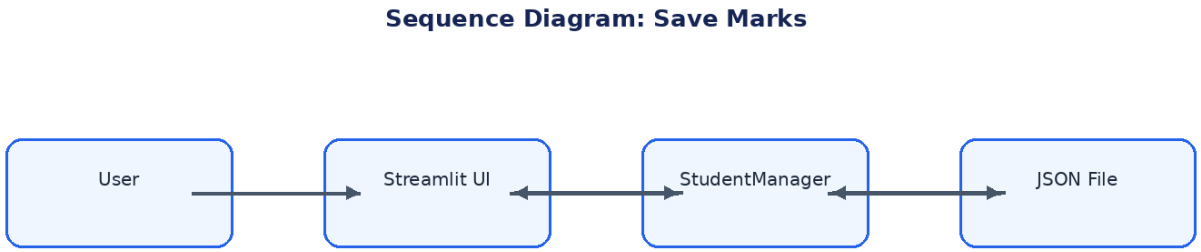
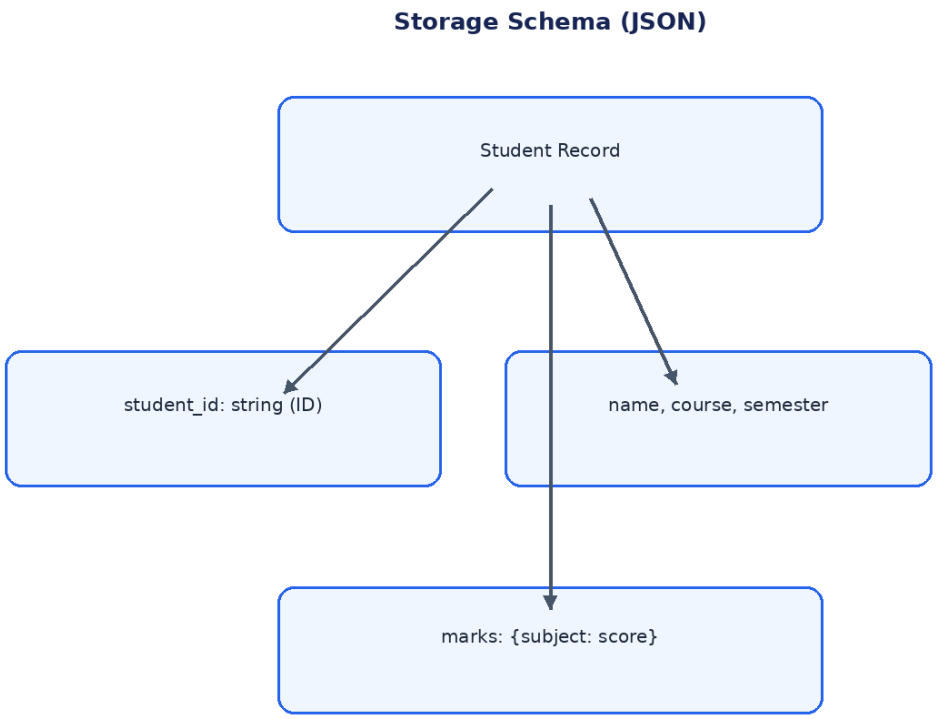


Figure 6. JSON storage schema



8. Storage Design

The file data/students.json stores a list of student objects. Each object contains student_id, name, course, semester, and marks. The marks field is a dictionary mapping a subject name to a numeric score. A separate relational database is not used, so the schema is represented as a JSON storage structure rather than a relational ER model.

Example structure (illustrative): student_id: "STU001"; name: "Sample Student"; course: "Integrated M.Tech AI"; semester: "1"; marks: {"Python": 85}. This example is documentation only; use your own test data when demonstrating the app.

9. Design Decisions and Rationale

• JSON was selected because it is human-readable and supported by Python's standard library.

• StudentManager centralizes CRUD-style record operations to avoid duplicating storage logic.

• marks_manager.py isolates result calculation so it can be tested independently.

• Streamlit was used for a quick browser-based interface without requiring separate HTML/CSS/JavaScript files.

• The command-line interface is retained as a simple alternative and demonstrates terminal input/output.

10. Implementation Details

File	Responsibility
app.py	Streamlit pages, forms, tables, and metrics
main.py	Command-line menu and program flow
student.py	Creates the student dictionary structure

File	Responsibility
student_manager.py	Adds, finds, updates, deletes records and updates marks
marks_manager.py	Calculates total, average percentage, and grade
report_manager.py	Prints student lists and individual reports in the CLI
file_handler.py	Loads and saves JSON records
validators.py	Reusable CLI input validation
tests/test_project.py	Unit tests for result calculations

11. Screenshots and Results

The Streamlit application provides Dashboard, Add Student, View Students, and Marks & Results pages. The dashboard shows record counts and an average percentage based on students with marks. The marks page shows subject scores and a result summary. Before submitting, capture actual screenshots from your running application and add them to your report if your instructor expects visual evidence. Do not use sample figures as proof of a live run.

12. Testing Approach

Automated unit tests were run with Python unittest for average/grade calculation, the no-marks case, and a failing grade. The latest test run completed three tests successfully. Manual checks should also verify adding a student, reopening the app to confirm persistence, entering marks, and deleting a record only after confirmation.

Automated test cases

Test	Scenario	Expected result
TC-1	Marks 80 and 90	Total 170, average 85, grade A
TC-2	No marks recorded	0% and grade Not available
TC-3	Single score of 25	Grade F

13. Challenges Faced

• Keeping the terminal version and browser-based version connected to the same JSON data file.

• Ensuring a student is selected before marks can be entered.

• Handling empty fields and duplicate student IDs without corrupting stored records.

• Separating result calculation from the user interface so it can be tested independently.

14. Learnings and Key Takeaways

• Using functions and classes to divide a program into understandable parts.

• Using dictionaries and lists to represent structured records.

• Reading and writing JSON files for persistence.

• Building a basic web UI with Streamlit forms, tables, sidebar navigation, and metrics.

• Writing small automated unit tests with unittest.

15. Future Enhancements

• Add a dedicated edit-student page in the Streamlit interface.

• Add search and filtering controls to the web interface.

• Add authentication and role-based permissions if the application is extended beyond a local demo.

• Add CSV export, backup, and import features.

• Use a database if the number of records grows significantly.

16. References

• Python documentation: https://docs.python.org/3/

• Streamlit documentation: https://docs.streamlit.io/

• Python unittest documentation: https://docs.python.org/3/library/unittest.html

• Project source code and its README.md / statement.md files.

Submission checklist

• Replace the cover-page name and roll-number placeholders before uploading.

• Run the Streamlit app from the project folder and capture genuine screenshots if required.

• Run `python -m unittest discover -s tests -v` before final submission.

• Create a public GitHub repository and submit its root URL, not a /tree or /blob URL.

• Upload this PDF separately to the course portal as instructed.